

TILE TYPE COUNTER SYSTEM

OVERVIEW

The Tile Type Counter System tracks different types of tiles that can be placed in the game. It does this with a bunch of tile trackers. Each tile tracker tracks a different type of tile. For example, the residence tile tracker keeps a list of all the residences. This system is managed by the script `TileTypeCounter.cs`, whose singleton sits of the Grid game object. Also heavily involved is the abstract class `TileTracker`, from which each kind of tile tracker inherits. The `TileTracker` class and all of the classes that inherit from it are at the bottom of the `TileTypeCounter.cs` file, which admittedly is not best practice. (They should probably have their own separate files, but I was too lazy to press the “new file” button)

USING TILE TRACKERS

You can access all tile trackers through the `TileTypeCounter` class, which is a singleton accessible at “`TileTypeCounter.current`”. You can access all the tiles stored by a tile tracker using the `GetAllTiles` function, which returns them as an array:

```
TileTypeCounter.current.ExampleTileTracker.GetAllTiles()
```

LIST OF TILE TRACKERS

These are all of the tile trackers currently in the game:

CarbonTileTracker

Keeps track of all the tiles on the active chunk that produce carbon emissions.

MoneyTileTracker

Tracks all of the tiles on the active chunk that have an annual income greater than 0.

AllTileTracker

Keeps track of all the tiles on the active chunk.

RoadTileTracker

Keeps track of all of the road tiles on the active chunk.

ResidenceTileTracker

Keeps track of all the residence tiles on the active chunk.

FactoryTileTracker

Keeps track of all the factory tiles on the active chunk.

WorkplaceTileTracker

Keeps track of all the tiles who require employees. Tracks all tiles like this across every area, not just on the active chunk.

CarbonCaptureTileTracker

Keeps track of all the carbon capture systems on the active chunk.

ActivatableBuildingTileTracker

Keeps track of all the buildings on the active chunk that need to be connected by roads to activate.

ActivatableTileTracker

Keeps track of all the tiles on the active chunk that can be activated, including both roads and buildings.

HOW TILE TRACKERS WORK

The class `TileTypeCounter`, which is accessible through a singleton, instantiates all of the tile trackers at the start of the game and adds them to its list of tile trackers, called `TileTrackers`. Each tile tracker inherits from the `TileTracker` class.

Whenever a tile is placed, the `GridManager` gives the `TileTypeCounter` the reference to the new tile and tells it to check if it should be added to any of the tile trackers. To do this, for each of the tile trackers in the `TileTrackers` list, `TileTypeCounter` runs `ExampleTileTracker.CheckTileForAddition(newTile)`. This will cause each tile tracker to check if the new tile is the kind of tile it's keeping track of. For example, the `ResidenceTileTracker` will check if the new tile is a residence. If the new tile is the kind of tile a given tile tracker is looking for, that tile tracker will add it to its own internal list of tiles.

By default, all the tile trackers add a listener at the beginning of the game for when the player switches areas. When that event occurs, the listener will call a function to reset the tile tracker. After resetting itself, each tile tracker will get a list of all the tiles on the new chunk from the grid manager and check to see if they should add them to its list. If so, then the tile tracker adds the tile to its list. That way, each tile tracker keeps track of the tiles on the active grid chunk only. However, tile trackers can be prevented from resetting themselves when switching between areas. To do this, you simply have to override the tile tracker's `bool resetThisTrackerOnChunkSwitch` to return false. (To see an example, look at the `WorkplaceTileTracker` class)

ADDING A NEW TILE TRACKER

1. To create a new tile tracker, first you have to add a new class for the tile tracker at the bottom of the `TileTypeCounter.cs` file. (Or you can move all the classes to separate files if you're in the mood to organize my messy code.) Be sure the new class inherits from `TileTracker`.
 - a. Add override methods for `CheckTileForAddition(Tile tile)` and `CheckTileForRemoval(Tile tile)` that determine if "tile" should be tracked by the tile tracker. If so, use `base.AddTile(tile)` or `base.RemoveTile(tile)`.
2. Add a declaration of the new tile tracker you created at the top of the `TileTypeCounter` class, like this: `"TileTracker ExampleTileTracker {get; set;}"`
3. Create a new instance of the tile tracker you created in the `Start` function of `TileTypeCounter` and add it to the `TileTrackers` list, like this:

```
ExampleTileTracker = new ExampleTileTrackerClass();  
  
TileTrackers.Add(ExampleTileTracker);
```

Note: The system for adding new tile trackers could definitely be easier. If you want to, feel free to clean it up.

INTERNAL API

These are some of the important public functions of the `TileTypeCounter` class, whose singleton is accessible at `TileTypeCounter.current`.

`void CheckTileTrackersForAddition(GameObject objectToCheck)`

Makes each tile tracker check if the input tile, “objectToCheck”, should be added by the tile tracker. Called by the `GridManager` when a new tile is placed.

`void CheckTileTrackersForRemoval(GameObject objectToCheck)`

Makes each tile tracker check if the input tile, “objectToCheck”, should be removed from the tile tracker. Called by the `GridManager` whenever a tile is deleted.

`Tile[] GetAllActivatedBuildings()`

Returns an array of all the buildings on the activated chunk that are currently activated. It does this by looking at the activatable building tracker, which tracks all of the buildings that can be activated. It returns all the activatable buildings that are in fact currently activated.

`Tile[] GetAllActivatedRoads()`

Returns a tile array of all the roads on the active grid chunk that are currently activated. It does this by getting all the tiles from the road tile tracker, which keeps track of all the roads, and then returning each one that is currently activated.

`TileTypeCounter.current.ExampleTileTracker.GetAllTiles()`

This technically isn't a public function of the `TileTypeTracker` class, but it's how you access all of the tiles tracked by `ExampleTileTracker`.

-Graydon Bruyn, May 2026